

DETECÇÃO DE TUMORES CEREBRAIS COM INTELIGÊNCIA ARTIFICIAL

Comparando MLP (Rede Neural Densa) e Vision Transformer

Nome: Luana Aparecida Neres Lopes

1. Introdução

Diagnóstico de tumores cerebrais por meio de imagens de Ressonância Magnética (RM) é uma tarefa importante, mas que exige experiência do profissional e pode levar tempo. Nesse contexto, técnicas de Inteligência Artificial podem auxiliar na classificação das imagens, tornando esse processo mais rápido e consistente.

Neste trabalho, foram realizados oito experimentos para classificar imagens de RM em quatro categorias: glioma, meningioma, tumor hipofisário e ausência de tumor. Para isso, foram comparadas duas abordagens: Redes Neurais Perceptron Multicamadas (MLP) e Vision Transformer (ViT). Cada técnica irá passar por 4 variações de arquitetura e de estratégia de regularização.

1.1 O dataset

Dataset: Brain Tumor MRI Dataset — kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset

Contém imagens de RM cerebral organizadas em quatro classes :

- Glioma — tumor maligno nas células de suporte do cérebro;
- Meningioma — tumor geralmente benigno nas membranas do cérebro;
- Tumor hipofisário (pituitary) — tumor na glândula hipófise;
- Sem tumor (notumor) — exames sem evidência de tumor.

Conjunto	Glioma	Meningioma	Sem tumor	Hipofisário	Total
Treino	1.400	1.400	1.400	1.400	5.600
Teste	400	400	400	400	1.600

As imagens foram divididas em conjuntos de treinamento, validação e teste. O treinamento foi realizado com as imagens de treino, 20% desse conjunto foi reservado para validação, e as imagens de teste foram utilizadas apenas para avaliar o desempenho final dos modelos.

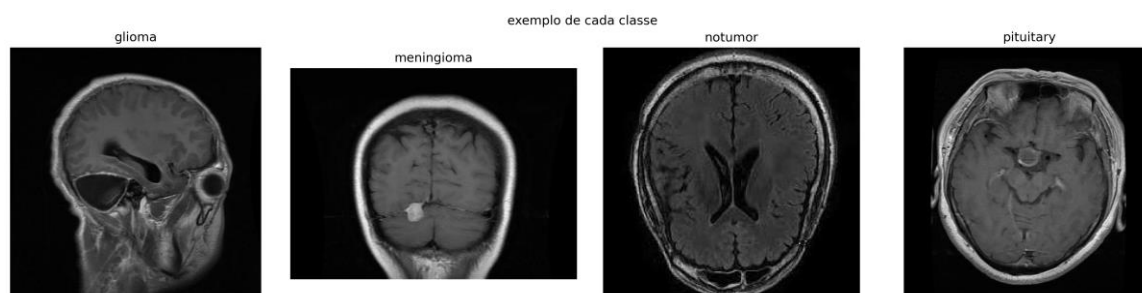


Figura 1 – imagens de RM cerebral utilizadas nos experimentos, ilustrando uma amostra de cada uma das quatro classes presentes no conjunto de dados.

1.2 Ferramentas utilizadas

- Python 3.10 no Google Colab com GPU T4;

- TensorFlow 2.20 / Keras — construção e treino da MLP e do Transformer;
- Scikit-Learn — cálculo das métricas;
- Kaggle API — download automatizado do dataset.

2. Metodologia

2.1 Pré-processamento

Antes do treinamento, todas as imagens passaram por um processo de preparação. Os valores dos pixels foram normalizados para o intervalo de 0 a 1, facilitando o aprendizado dos modelos. Para a MLP, as imagens foram redimensionadas para 64×64 pixels. Em seguida, cada imagem foi transformada em um único vetor com 12.288 valores, que corresponde à entrada da rede neural.

Para o Vision Transformer, as imagens foram redimensionadas para 80×80 pixels e divididas em 100 pequenos blocos (patches) de 8×8 pixels. Esses blocos são analisados pelo modelo para identificar características importantes da imagem.

Além disso, o conjunto de treinamento foi dividido em 80% para treinamento e 20% para validação, permitindo acompanhar o desempenho dos modelos durante o processo de aprendizagem.

2.2 Data Augmentation

Nos experimentos 7 e 8 foi utilizada a técnica de Data Augmentation, que cria pequenas variações nas imagens durante o treinamento sem alterar sua classe.

As transformações aplicadas foram:

- rotação de até 20° ;
- deslocamento horizontal e vertical de até 15%;
- zoom de até 15%;
- espelhamento horizontal;
- pequenas variações no brilho.

O objetivo dessa técnica é aumentar a variedade de imagens apresentadas ao modelo, reduzindo o risco de overfitting, ou seja, evitando que o modelo memorize apenas as imagens de treinamento e tenha dificuldade para classificar novas imagens.

2.3 Métricas de avaliação

Para avaliar o desempenho dos modelos foram utilizadas quatro métricas, calculadas com a biblioteca Scikit-Learn. Foi utilizada a média macro, garantindo que todas as quatro classes tivessem a mesma importância na avaliação.

- Acurácia: indica a porcentagem de classificações realizadas corretamente pelo modelo.
- Precisão: mede quantas das classificações feitas pelo modelo estavam corretas.
- Recall: indica a capacidade do modelo de identificar corretamente as imagens de cada classe.
- F1-Score: combina Precisão e Recall em uma única métrica, sendo útil para avaliar o desempenho geral do modelo.

Essas métricas foram calculadas automaticamente ao final de cada experimento e utilizadas para comparar o desempenho entre as diferentes configurações da MLP e do Vision Transformer.

3. Técnicas utilizadas

3.1 MLP — Multilayer Perceptron

A MLP (Perceptron Multicamadas) foi utilizada como modelo base para a classificação das imagens de ressonância magnética. Antes do treinamento, cada imagem foi convertida em um vetor de valores, que é processado por camadas totalmente conectadas. Diferentes configurações foram avaliadas, variando o número de neurônios, a quantidade de camadas ocultas e a aplicação de técnicas de regularização, como Dropout, L2 e Batch Normalization. Essas variações tiveram como objetivo analisar o impacto de cada técnica no desempenho do modelo

3.2 Vision Transformer

O Vision Transformer (ViT) foi utilizado como uma abordagem mais moderna para classificação de imagens. Nesse modelo, a imagem é dividida em pequenos blocos (*patches*), que são processados por mecanismos de atenção capazes de capturar relações entre diferentes regiões da imagem. Nos experimentos, foram avaliadas diferentes configurações do modelo, alterando parâmetros como o número de blocos Transformer, quantidade de cabeças de atenção (*heads*), dimensão dos embeddings, aplicação de regularização L2 e uso de Data Augmentation, permitindo comparar o desempenho de arquiteturas mais simples e mais complexas.

4. Os 8 experimentos realizados

4.1 Grupo 1 — MLP (Experimentos 1 a 4)

Experimento 1 — MLP simples

```
# Experimento 1: MLP simples
treino, val, teste = criar_geradores(IMG_SIZE_MLP, augmentation=False)

modelo1 = criar_mlp(neuronios=256, n_camadas=2, dropout=0.0, l2=0.0, batchnorm=False)
modelo1.summary()

t0 = time.time()
hist1 = modelo1.fit(treino, validation_data=val, epochs=EPOCAS_MLP, verbose=1)
t1 = time.time()

avaliar(modelo1, teste, "1) MLP simples (256 neurônios, sem regularização)", hist1)
```

Experimento 1 – MLP Simples: Modelo composto por duas camadas ocultas de 256 neurônios cada, sem aplicação de técnicas de regularização. Esse experimento foi utilizado como referência para avaliar o efeito das modificações introduzidas nos demais modelos.

Parâmetro	Valor
Neurônios	256
Camadas ocultas	2
Dropout	Não
Regularização L2	Não
Batch Normalization	Não

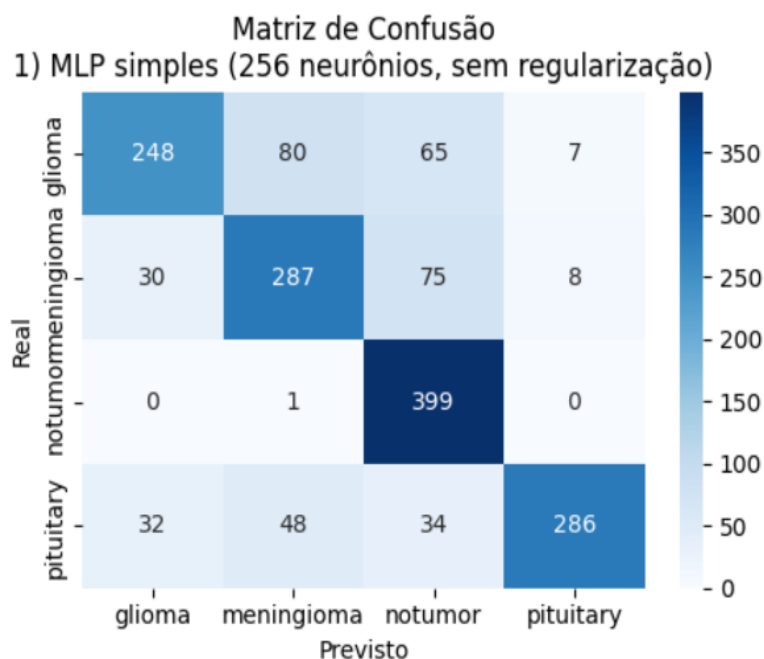


Figura 2 – Matriz de confusão do Experimento 1 MLP simples. A matriz apresenta as classificações corretas e incorretas realizadas pelo modelo para cada uma das quatro classes.

Observa-se que a Loss de treinamento diminuiu continuamente, enquanto a Loss de validação passou a oscilar e aumentou nas últimas épocas. Além disso, a acurácia de treinamento atingiu valores próximos de 100%, enquanto a acurácia de validação permaneceu em torno de 85%. Esse comportamento indica a ocorrência de overfitting, ou seja, o modelo aprendeu muito bem os dados de treinamento, mas apresentou menor capacidade de generalização para dados não vistos.

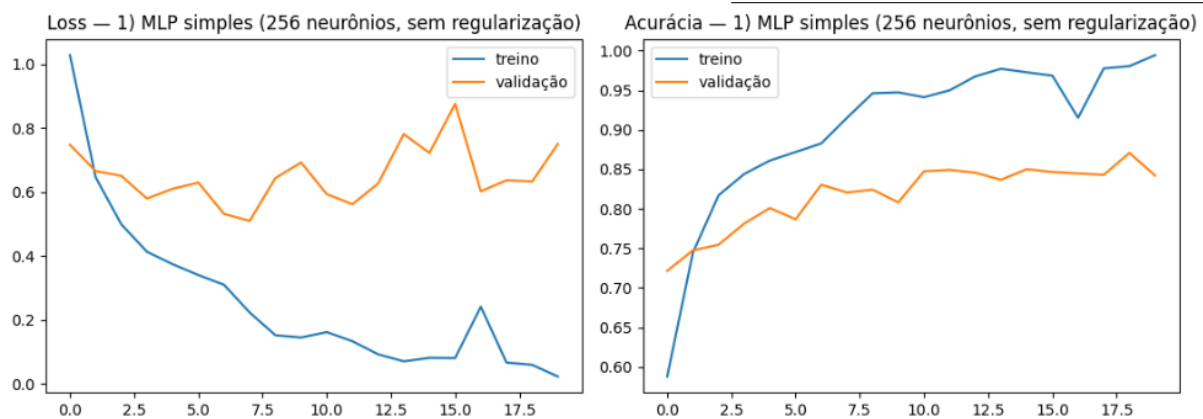


Figura 3 - Curvas de Loss e Acurácia do Experimento 1 MLP simples. As curvas mostram a evolução da perda e da acurácia durante o treinamento e a validação do modelo.

A matriz de confusão mostra que o modelo obteve excelente desempenho na classe No Tumor, com 399 acertos em 400 imagens. As classes Meningioma e Pituitary também apresentaram bons resultados. Entretanto, a classe Glioma foi a que apresentou maior número de erros, sendo frequentemente confundida com Meningioma e No Tumor, indicando maior dificuldade do modelo em diferenciar essas categorias.

Experimento 2 — MLP + Dropout

```
#Experimento 2: MLP + Dropout

treino, val, teste = criar_geradores(IMG_SIZE_MLP, augmentation=False)

modelo2 = criar_mlp(neuronios=256, n_camadas=2, dropout=0.4, l2=0.0, batchnorm=False)

t0 = time.time()
hist2 = modelo2.fit(treino, validation_data=val, epochs=EPOCAS_MLP, verbose=1,
                    callbacks=[EarlyStopping(monitor='val_loss', patience=5,
                                             restore_best_weights=True)])
t1 = time.time()

avaliar(modelo2, teste, "2) MLP + Dropout (0,4)", hist2)
```

-Foi utilizada uma MLP composta por duas camadas ocultas de 256 neurônios cada, com aplicação de Dropout de 40% após cada camada oculta. O objetivo foi avaliar o impacto

dessa técnica de regularização na redução do overfitting e na capacidade de generalização do modelo.

Parâmetro	Valor
Neurônios	256
Camadas ocultas	2
Dropout	40%
Regularização L2	Não
Batch Normalization	Não

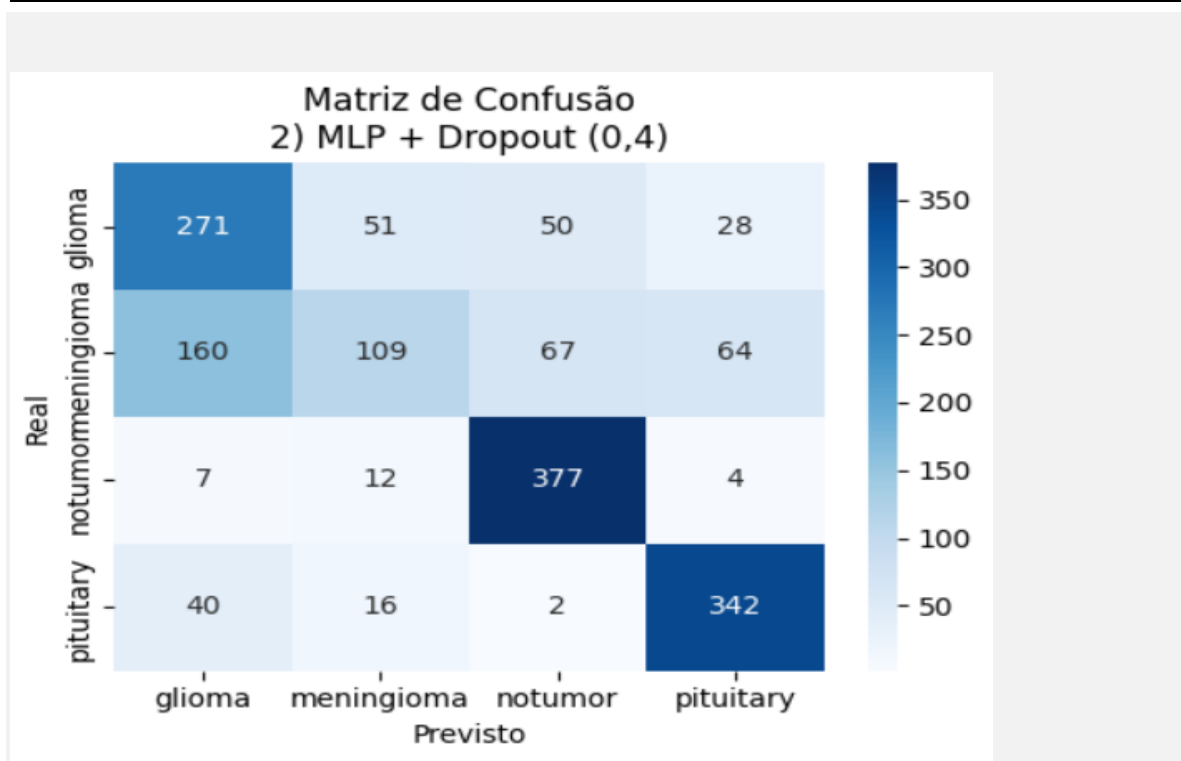


Figura 4 -Matriz de confusão do Experimento 2 (MLP com Dropout de 40%). A matriz apresenta a distribuição das classificações corretas e incorretas realizadas pelo modelo para cada classe.

A matriz de confusão mostra que o modelo apresentou melhor desempenho na classificação das classes No Tumor e Pituitary, enquanto a maior dificuldade ocorreu na distinção entre Glioma e Meningioma, que concentraram a maior parte dos erros de classificação. Em comparação ao modelo sem regularização, o uso do Dropout reduziu o overfitting, embora ainda existam dificuldades na separação dessas duas classes

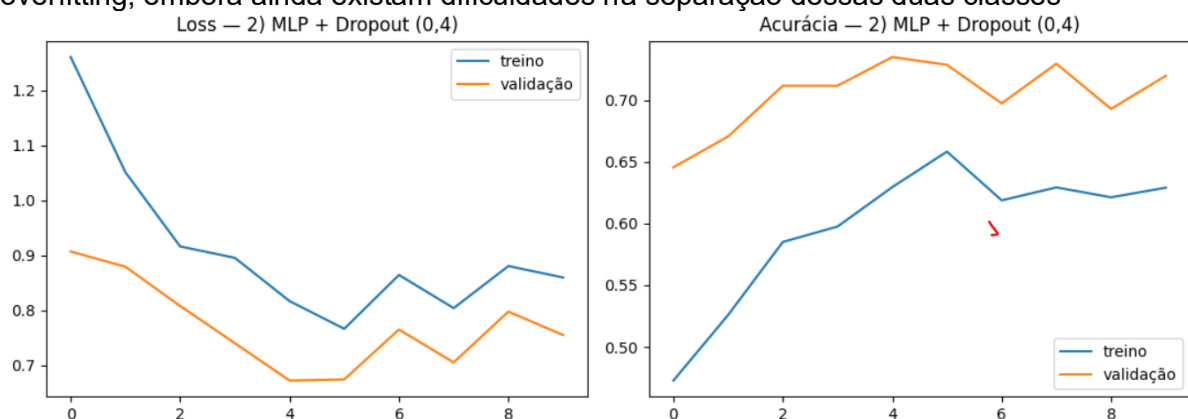


Figura 5 -Curvas de Loss e Acurácia do Experimento 2 (MLP com Dropout de 40%)- As

curvas apresentam a evolução da perda (Loss) e da acurácia nos conjuntos de treinamento e validação ao longo das épocas.

Observa-se que a perda de treinamento e validação diminuiu durante o treinamento, enquanto a acurácia aumentou gradativamente. As curvas permaneceram próximas entre si, indicando que a aplicação do **Dropout** contribuiu para reduzir o overfitting e melhorar a capacidade de generalização do modelo em relação ao experimento anterior.

Experimento 3 — MLP + Dropout + Regularização L2

```
# Experimento 3: MLP + Dropout + L2
treino, val, teste = criar_geradores(IMG_SIZE_MLP, augmentation=False)

modelo3 = criar_mlp(neuronios=256, n_camadas=2, dropout=0.4, l2=0.001, batchnorm=False)

t0 = time.time()
hist3 = modelo3.fit(treino, validation_data=val, epochs=EPOCAS_MLP, verbose=1,
                    callbacks=[EarlyStopping(monitor='val_loss', patience=5,
                                             restore_best_weights=True)])
t1 = time.time()

avaliar(modelo3, teste, "3) MLP + Dropout (0,4) + L2 (0,001)", hist3)
```

Foi utilizada uma MLP composta por **duas camadas ocultas de 256 neurônios cada**, com aplicação de **Dropout de 40%** e **regularização L2**. Enquanto o Dropout reduz o overfitting ao desativar aleatoriamente parte dos neurônios durante o treinamento, a regularização L2 penaliza pesos muito elevados, favorecendo modelos com maior capacidade de generalização. O objetivo deste experimento foi avaliar o efeito da combinação dessas duas técnicas no desempenho da rede.

Parâmetro	Valor
Neurônios	256
Camadas ocultas	2
Dropout	40%
Regularização L2	$\lambda = 0,001$
Batch Normalization	Não

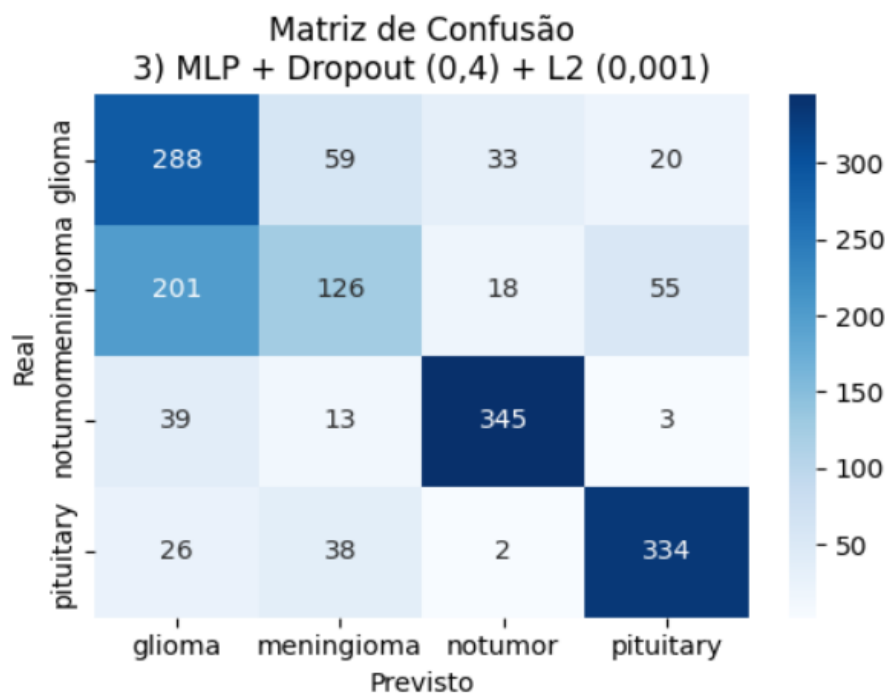


Figura 6 Matriz de confusão do Experimento 3 (MLP com Dropout de 40%+ L2). A matriz apresenta a distribuição das classificações corretas e incorretas realizadas pelo modelo para cada classe.

A matriz de confusão mostra que o modelo apresentou melhor desempenho na classificação das classes No Tumor e Pituitary, enquanto a maior dificuldade ocorreu na distinção entre Glioma e Meningioma, que concentraram a maior parte dos erros de classificação. Em comparação ao modelo sem regularização, o uso do Dropout reduziu o overfitting, embora ainda existam dificuldades na separação dessas duas classes. Vale destacar que, apesar de o overfitting ter sido controlado, este foi o experimento com menor F1-Score, o que reforça a hipótese de que 20 épocas foram insuficientes para modelos com regularização mais intensa.

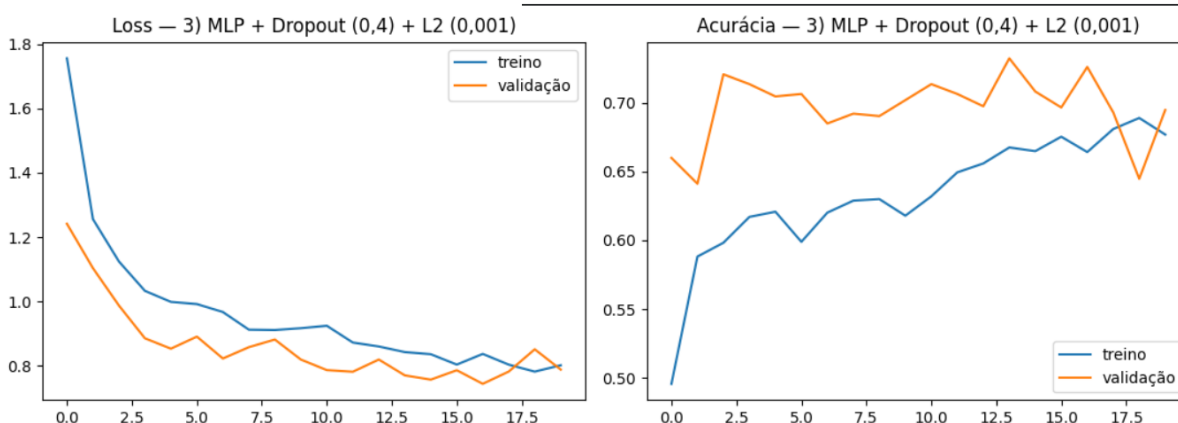


Figura 7 -Curvas de Loss e Acurácia do Experimento 3 , MLP com Dropout de 40% e

regularização L2. As curvas apresentam a evolução da perda e da acurácia nos conjuntos de treinamento e validação durante o treinamento.

Observa-se que a **Loss** diminuiu de forma gradual ao longo das épocas, enquanto a **acurácia** aumentou tanto no treinamento quanto na validação. As curvas permaneceram relativamente próximas, indicando que a combinação entre **Dropout** e **regularização L2** ajudou a controlar o overfitting e proporcionou um treinamento mais estável.

Experimento 4 — MLP maior + BatchNormalization

```
# Experimento 4: MLP maior + BatchNormalization
treino, val, teste = criar_geradores(IMG_SIZE_MLP, augmentation=False)

modelo4 = criar_mlp(neuronios=512, n_camadas=3, dropout=0.3, l2=0.0, batchnorm=True)

t0 = time.time()
hist4 = modelo4.fit(treino, validation_data=val, epochs=EPOCAS_MLP, verbose=1,
                    callbacks=[EarlyStopping(monitor='val_loss', patience=5,
                                             restore_best_weights=True)])
t1 = time.time()

avaliar(modelo4, teste, "4) MLP maior (512 neurônios, 3 camadas) + BatchNorm", hist4)
```

Neste experimento, a arquitetura da MLP foi ampliada para três camadas ocultas de 512 neurônios cada, tornando a rede mais robusta para aprender os padrões das imagens. Também foi utilizada a técnica de batch normalization, que ajuda a tornar o treinamento mais estável, além de Dropout de 30%, com o objetivo de reduzir o overfitting. Esse experimento buscou avaliar se uma rede maior, combinada com essas técnicas, seria capaz de melhorar o desempenho da classificação.

Parâmetro	Valor
Neurônios	512
Camadas ocultas	3
Dropout	30%
Regularização L2	Não
Batch Normalization	Sim

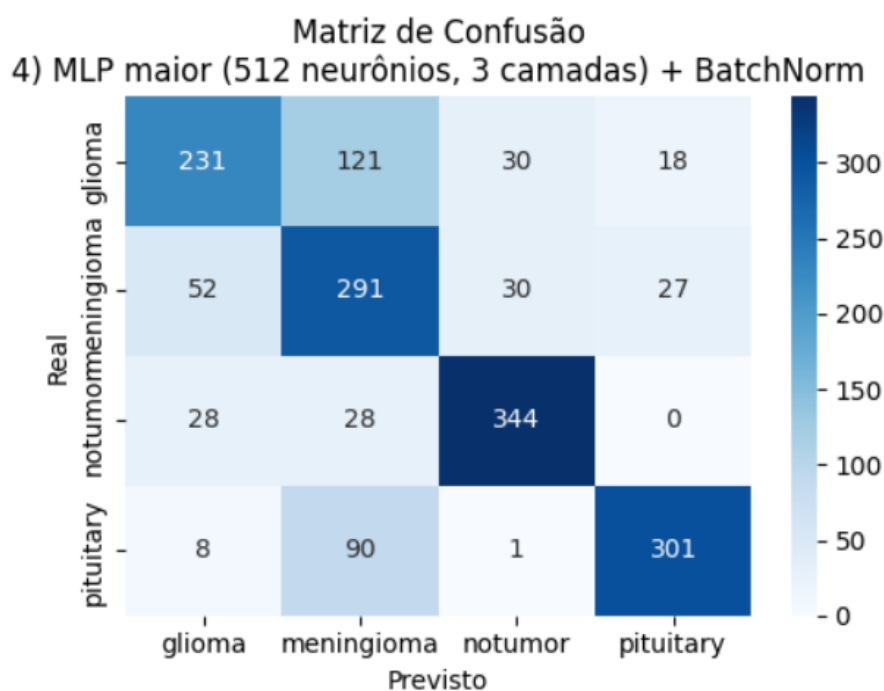


Figura 8 - Matriz de confusão do Experimento 4 (MLP maior com Batch Normalization). A matriz apresenta as classificações corretas e incorretas realizadas pelo modelo para cada classe.

A matriz de confusão mostra que o modelo manteve bom desempenho na classificação das classes No Tumor, Meningioma e Pituitary. No entanto, a classe Glioma continuou apresentando maior número de erros, sendo frequentemente confundida com Meningioma. Esses resultados indicam que o aumento da rede e a utilização do Batch Normalization não foram suficientes para eliminar as dificuldades na distinção entre essas classes.

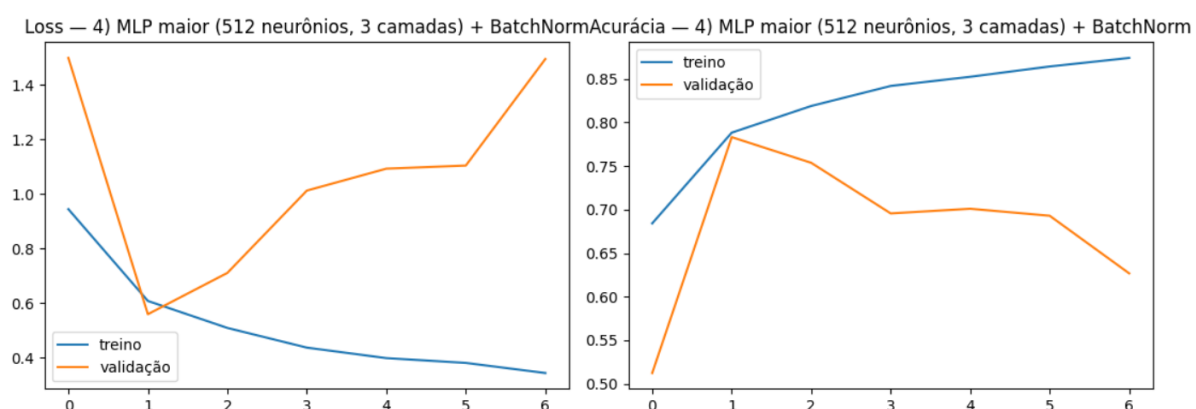


Figura 9-Curvas de Loss e Acurácia do Experimento 4 MLP maior com Batch Normalization- As curvas apresentam a evolução da perda e da acurácia nos conjuntos de treinamento e validação durante o treinamento do modelo.

Observa-se que a Loss de treinamento diminuiu continuamente, enquanto a Loss de validação aumentou após as primeiras épocas. Da mesma forma, a acurácia de treinamento continuou crescendo, enquanto a acurácia de validação apresentou queda. Esse comportamento indica que, apesar do aumento da capacidade da rede, o modelo voltou a

apresentar overfitting, mostrando que a arquitetura maior não resultou em melhor generalização

4.2 Grupo 2 — Vision Transformer (Experimentos 5 a 8)

Experimento 5 — Transformer simples (ponto de controle)

```
# Experimento 5 | transformer simples
treino_tr, val_tr, teste_tr = criar_geradores(IMG_SIZE_TR, augmentation=False)

modelo5 = criar_transformer(n_blocos=2, dim=64, heads=4, dropout=0.1, l2=0.0)
modelo5.summary()

t0 = time.time()
hist5 = modelo5.fit(treino_tr, validation_data=val_tr, epochs=EPOCHS_TR, verbose=1)
t1 = time.time()

avaliar(modelo5, teste_tr, "5) Transformer simples (2 blocos, 4 heads, sem L2)", hist5)
```

Neste experimento foi utilizado um **Vision Transformer (ViT)** com uma configuração básica, composta por **2 blocos Transformer**, **4 cabeças de atenção**, dimensão dos *embeddings* igual a **64** e **Dropout de 10%**. Não foi aplicada regularização L2. Esse modelo foi utilizado como **ponto de partida** para comparar o impacto das alterações realizadas nos experimentos seguintes com a arquitetura Vision Transformer.

Parâmetro	Valor
Blocos Transformer	2
Cabeças de atenção (Heads)	4
Dimensão dos embeddings	64
Dropout	10%
Regularização L2	Não

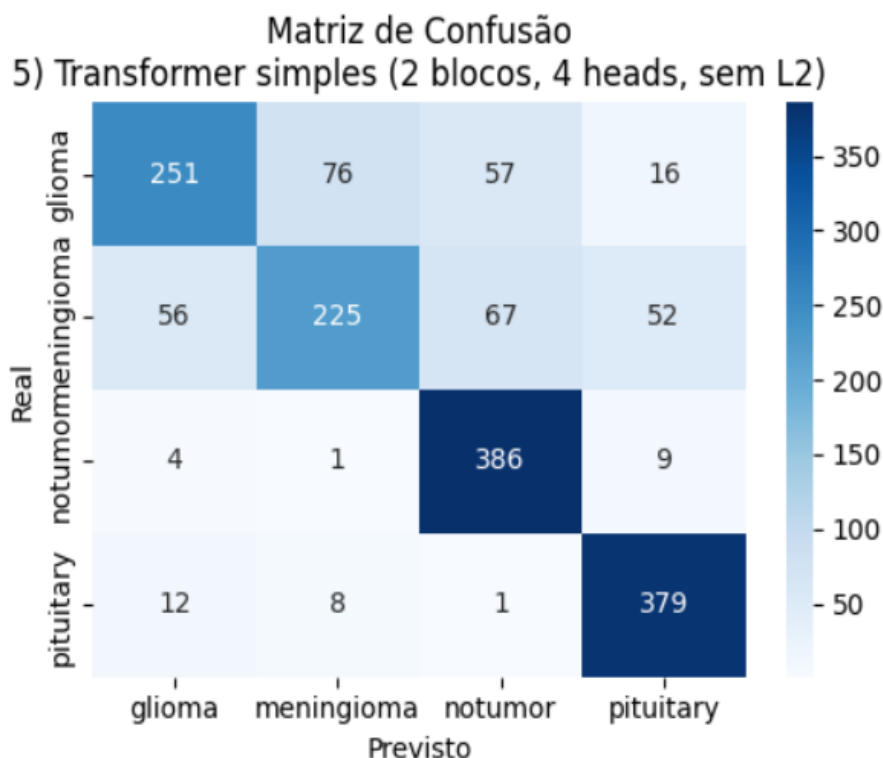


Figura 10- Matriz de confusão do Experimento 5 Vision Transformer simples. A matriz apresenta as classificações corretas e incorretas realizadas pelo modelo para cada classe.

A matriz de confusão mostra que o modelo apresentou excelente desempenho na classificação das classes **No Tumor** e **Pituitary**, que obtiveram o maior número de acertos. As principais dificuldades ocorreram entre as classes **Glioma** e **Meningioma**, que apresentaram algumas classificações incorretas entre si. No geral, o Vision Transformer simples demonstrou bom desempenho e serviu como base para comparação com os demais experimentos.

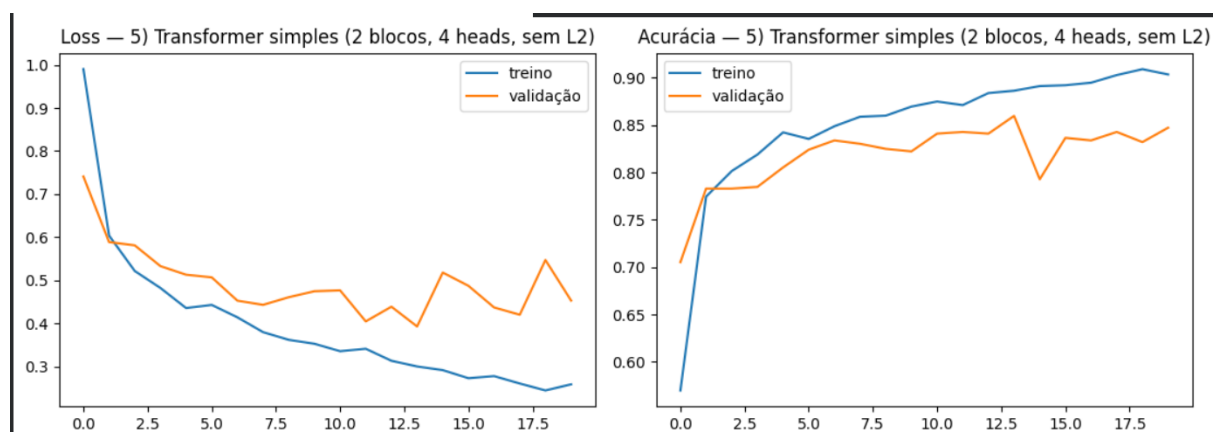


Figura 11 - Curvas de Loss e Acurácia do Experimento 5 (Vision Transformer simples). As curvas apresentam a evolução da perda e da acurácia nos conjuntos de treinamento e validação durante o treinamento do modelo.

Observa-se que a Loss diminuiu de forma contínua ao longo das épocas, enquanto a acurácia aumentou tanto no treinamento quanto na validação. As curvas permaneceram

próximas durante todo o treinamento, indicando que o modelo apresentou boa capacidade de generalização e baixo índice de overfitting, mesmo utilizando uma configuração mais simples do Vision Transformer.

Experimento 6 — Transformer + L2

```
#experimento 6: Transformer + L2

treino_tr, val_tr, teste_tr = criar_geradores(IMG_SIZE_TR, augmentation=False)

modelo6 = criar_transformer(n_blocos=2, dim=64, heads=4, dropout=0.1, l2=0.001)

t0 = time.time()
hist6 = modelo6.fit(treino_tr, validation_data=val_tr, epochs=EPOCAS_TR, verbose=1,
                    callbacks=[EarlyStopping(monitor='val_loss', patience=5,
                                             restore_best_weights=True)])
t1 = time.time()

avaliar(modelo6, teste_tr, "6) Transformer + L2 (0,001)", hist6)
```

Neste experimento foi utilizado o mesmo Vision Transformer do experimento anterior, porém foi adicionada a regularização L2. Essa técnica ajuda a evitar que o modelo se ajuste demais aos dados de treinamento, buscando melhorar sua capacidade de generalização. O objetivo foi verificar se essa alteração proporcionaria um melhor desempenho e comparar seus resultados com os obtidos pela MLP utilizando a mesma técnica.

Parâmetro	Valor
Blocos Transformer	2
Cabeças de atenção	4
Dimensão	64
Dropout	10%
Regularização L2	Sim ($\lambda = 0,001$)

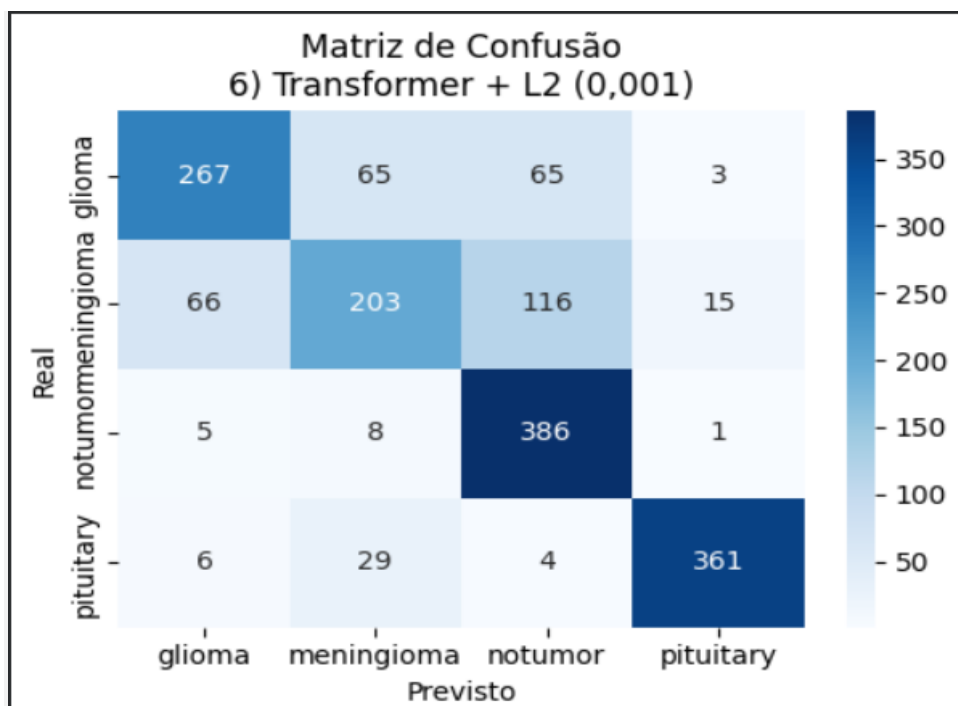


Figura 12 - Matriz de confusão do Experimento 6 (Vision Transformer com regularização L2). A matriz apresenta as classificações corretas e incorretas realizadas pelo modelo para cada classe.

A matriz de confusão mostra que o modelo apresentou bom desempenho na classificação das classes **No Tumor** e **Pituitary**, que obtiveram o maior número de acertos. As maiores dificuldades ocorreram entre as classes **Glioma** e **Meningioma**, que continuaram sendo confundidas em alguns casos. Mesmo assim, o Vision Transformer apresentou um desempenho consistente, indicando que a aplicação da regularização L2 contribuiu para melhorar a generalização do modelo.

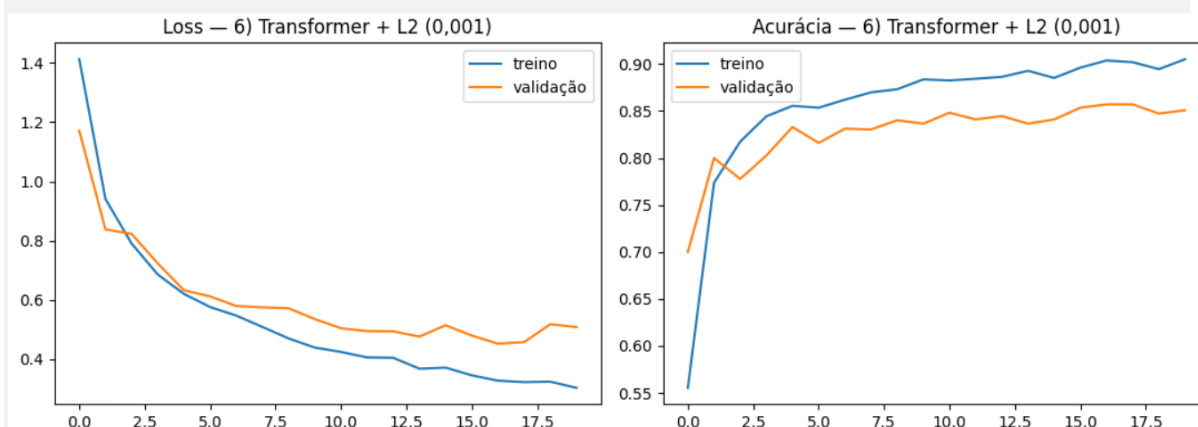


Figura 13 - Curvas de Loss e Acurácia do Experimento 6 (Vision Transformer com regularização L2). As curvas mostram a evolução da perda e da acurácia nos conjuntos de treinamento e validação durante o treinamento do modelo.

Observa-se que a Loss diminuiu continuamente ao longo das épocas, enquanto a acurácia aumentou tanto no treinamento quanto na validação. As curvas permaneceram próximas durante todo o treinamento, indicando que a regularização L2 contribuiu para um treinamento mais estável e para uma boa capacidade de generalização, sem sinais significativos de overfitting.

Experimento 7 — Transformer + L2 + Data Augmentation

```
# Experimento 7: Transformer + L2 + Data Augmentation

treino_tr_aug, val_tr_aug, teste_tr_aug = criar_geradores(IMG_SIZE_TR, augmentation=True)

modelo7 = criar_transformer(n_blocos=2, dim=64, heads=4, dropout=0.1, l2=0.001)

t0 = time.time()
hist7 = modelo7.fit(treino_tr_aug, validation_data=val_tr_aug, epochs=EPOCAS_TR,
                    verbose=1,
                    callbacks=[EarlyStopping(monitor='val_loss', patience=5,
                                             restore_best_weights=True)])
t1 = time.time()

avaliar(modelo7, teste_tr_aug, "7) Transformer + L2 + Augmentation", hist7)
```

Neste experimento foi utilizado o mesmo Vision Transformer do experimento anterior, acrescentando a técnica de **Data Augmentation** durante o treinamento. Essa técnica cria diferentes versões das imagens, realizando pequenas alterações como rotações, zoom e deslocamentos. O objetivo foi aumentar a variedade dos dados de treinamento, reduzindo a memorização das imagens e melhorando a capacidade de generalização do modelo.

Parâmetro	Valor
Blocos Transformer	2
Cabeças de atenção	4
Dimensão	64
Dropout	10%
Regularização L2	Sim ($\lambda = 0,001$)
Data Augmentation	Sim

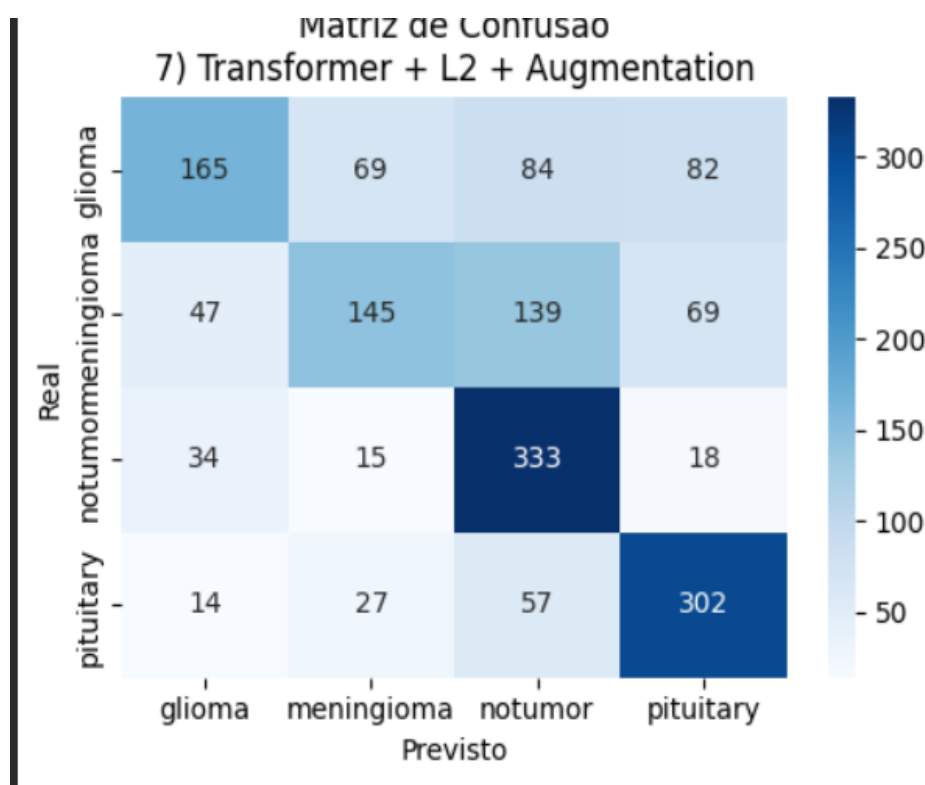


Figura 14 - Matriz de confusão do Experimento 7 Vision Transformer com L2 e Data Augmentation. A matriz apresenta as classificações corretas e incorretas realizadas pelo modelo para cada classe.

A matriz de confusão mostra que o modelo obteve melhor desempenho na classificação das classes No Tumor e Pituitary. Entretanto, ainda apresentou dificuldades para diferenciar as classes Glioma e Meningioma, que concentraram a maior parte dos erros de classificação. Embora o uso do Data Augmentation tenha contribuído para melhorar a capacidade de generalização do modelo, essas classes continuaram sendo as mais desafiadoras para a classificação.

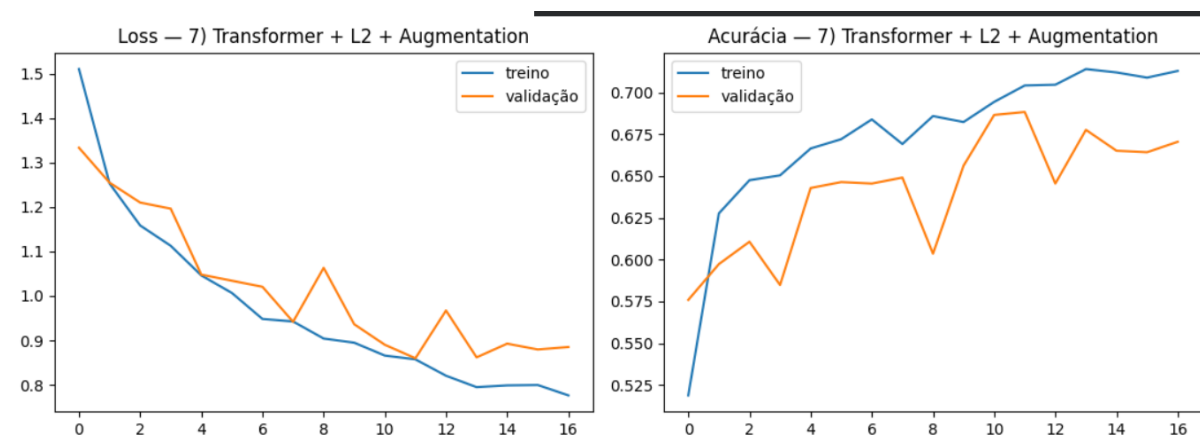


Figura 15 - Curvas de Loss e Acurácia do Experimento 7 (Vision Transformer com L2 e Data Augmentation). As curvas mostram a evolução da perda e da acurácia nos conjuntos de treinamento e validação durante o treinamento do modelo.

Observa-se que a **Loss** diminuiu ao longo das épocas e a **acurácia** apresentou crescimento tanto no treinamento quanto na validação. Apesar de pequenas oscilações, as curvas

permaneceram próximas, indicando que o uso do **Data Augmentation** ajudou a tornar o treinamento mais estável e reduziu a tendência de overfitting.

Experimento 8 — Transformer maior + L2 + Augmentation

```
# Experimento 8: Transformer maior + L2
modelo8 = criar_transformer(n_blocos=4, dim=96, heads=8, dropout=0.2, l2=0.001)

t0 = time.time()
hist8 = modelo8.fit(treino_tr_aug, validation_data=val_tr_aug, epochs=EPOCAS_TR,
                    verbose=1,
                    callbacks=[EarlyStopping(monitor='val_loss', patience=5,
                                             restore_best_weights=True)])
t1 = time.time()

avaliar(modelo8, teste_tr_aug,
        "8) Transformer maior (4 blocos, 8 heads) + L2 + Augmentation", hist8)
```

Epoch 1/20

Neste experimento foi utilizada uma versão mais robusta do Vision Transformer, aumentando a quantidade de blocos de atenção, o número de cabeças de atenção e a dimensão do modelo. Além disso, foram mantidas a regularização L2 e o Data Augmentation utilizados no experimento anterior. O objetivo foi verificar se uma arquitetura maior seria capaz de melhorar o desempenho da classificação das imagens.

Parâmetro	Valor
Blocos Transformer	4
Cabeças de atenção	8
Dimensão	96
Dropout	20%
Regularização L2	Sim ($\lambda = 0,001$)
Data Augmentation	Sim

Matriz de Confusão
8) Transformer maior (4 blocos, 8 heads) + L2 + Augmentation

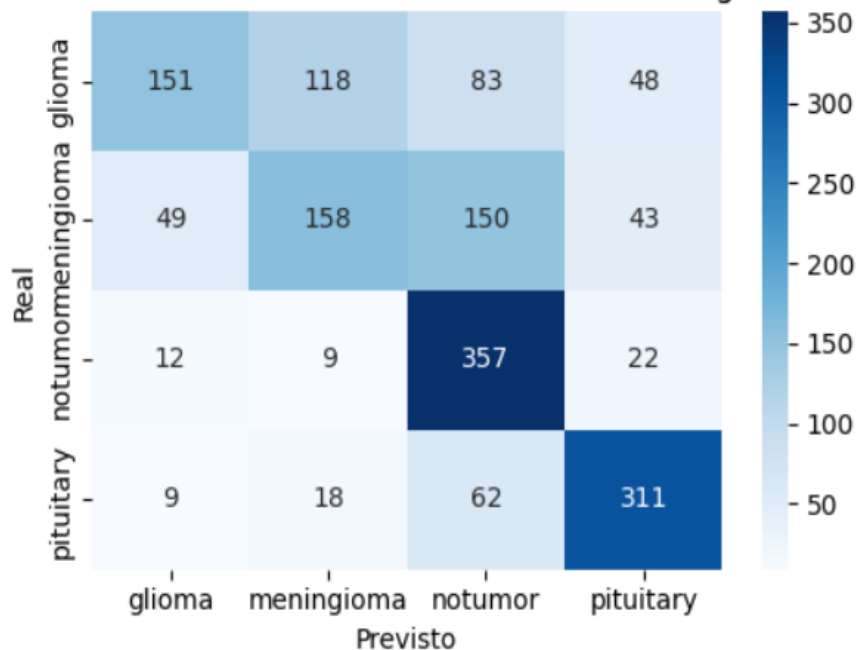


Figura 16 - Matriz de confusão do Experimento 8 (Vision Transformer maior com L2 e Data Augmentation). A matriz apresenta as classificações corretas e incorretas realizadas pelo modelo para cada classe.

A matriz de confusão mostra que o modelo apresentou melhor desempenho na classificação das classes No Tumor e Pituitary, que registraram o maior número de acertos. As maiores dificuldades permaneceram na diferenciação entre Glioma e Meningioma, que concentraram a maior parte das classificações incorretas. Apesar disso, a arquitetura maior manteve um desempenho consistente e demonstrou boa capacidade de classificação das imagens do conjunto de dados.

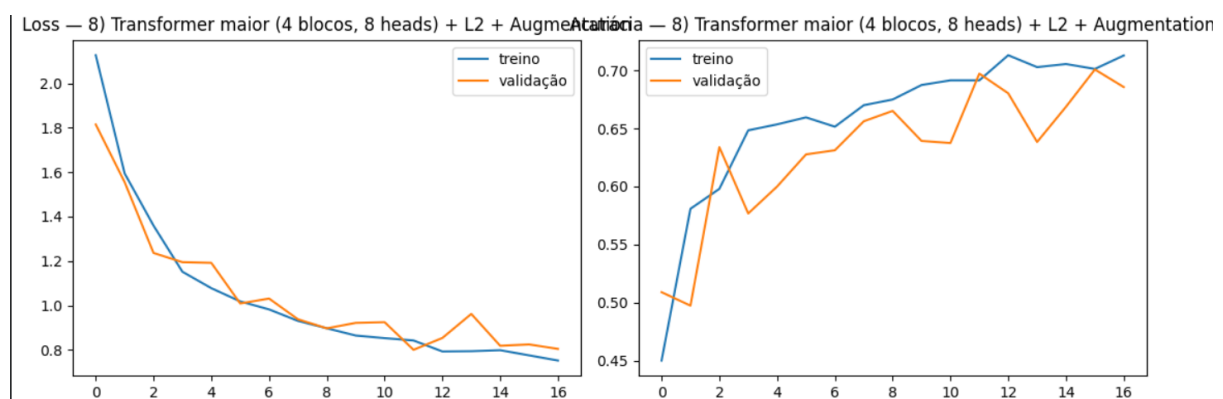


Figura 17 -Curvas de Loss e Acurácia do Experimento 8 (Vision Transformer maior com L2 e Data Augmentation). As curvas apresentam a evolução da perda e da acurácia nos conjuntos de treinamento e validação durante o treinamento do modelo.

Observa-se que a **Loss** diminuiu ao longo das épocas e a **acurácia** aumentou tanto no treinamento quanto na validação. As curvas permaneceram relativamente próximas durante o treinamento, indicando que o modelo apresentou boa capacidade de generalização, mesmo utilizando uma arquitetura maior. As pequenas oscilações observadas na validação são esperadas durante o processo de treinamento e não comprometem o desempenho geral do modelo.

5. Resultados

Todos os valores são reais, obtidos no conjunto de teste (1.600 imagens nunca vistas durante o treino), rodando no Google Colab com GPU T4.

5.1 Tabela de resultados

Exp.	Experimento	Acurácia	Precisão	Recall	F1-Score
1	MLP simples	0,8169	0,8233	0,8169	0,8127
2	MLP + Dropout	0,7131	0,7183	0,7131	0,6988
3	MLP + Dropout + L2	0,6575	0,6592	0,6575	0,6255
4	MLP maior + BatchNorm	0,6637	0,6810	0,6638	0,6554
5	Vision Transformer simples	0,7919	0,7946	0,7919	0,7843
6	Vision Transformer + L2	0,7606	0,7669	0,7606	0,7536
7	Vision Transformer + L2 + Augmentation	0,5906	0,5964	0,5906	0,5732
8	Vision Transformer maior	0,6106	0,6214	0,6106	0,5923

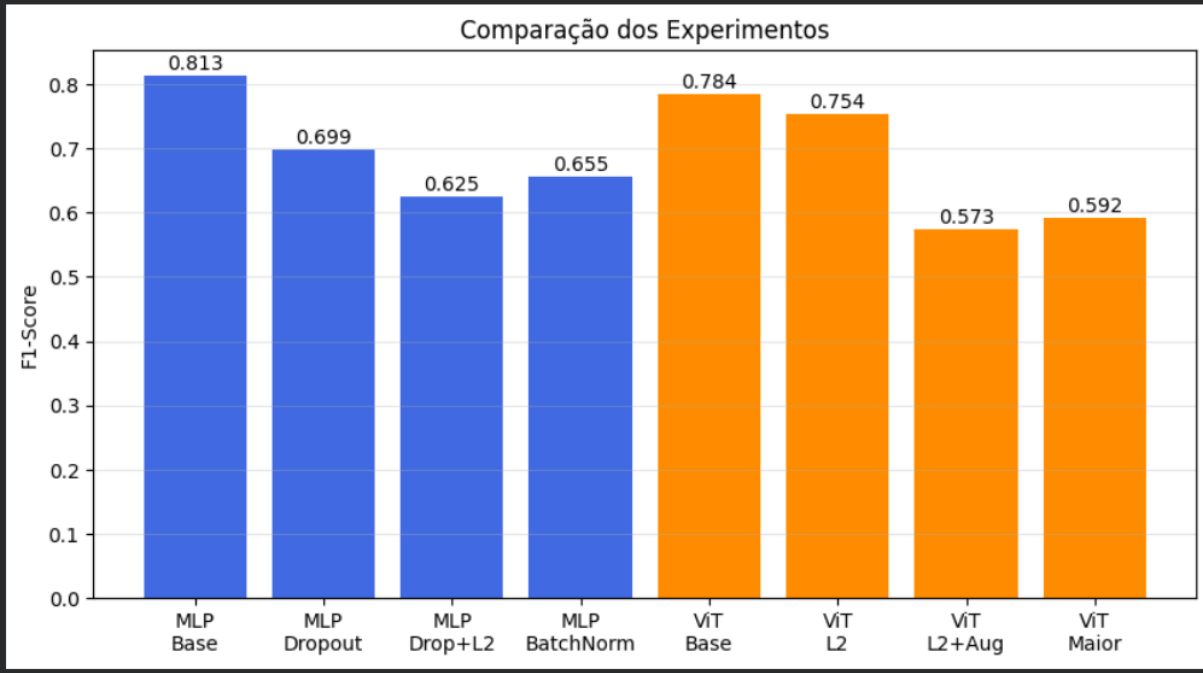


Figura 18 -Comparação do F1-Score obtido em cada experimento. O gráfico apresenta o desempenho dos modelos MLP e Vision Transformer nas diferentes configurações avaliadas.

Tabela ordenada por F1-Score.

5.2 Análise dos resultados

O melhor desempenho foi obtido no **Experimento 1 (MLP simples)**, que alcançou **F1-Score de 0,8127** e **acurácia de 0,8169**. Entre os modelos baseados em Vision Transformer, o melhor resultado foi o do **Experimento 5 (Transformer simples)**, com **F1-Score de 0,7843**.

De forma geral, observou-se que as versões mais simples dos modelos apresentaram melhor desempenho do que aquelas que utilizaram técnicas adicionais de regularização. Na MLP, a inclusão de **Dropout**, **regularização L2** e **Batch Normalization** não resultou em melhorias, apresentando valores de F1-Score inferiores ao modelo base. O mesmo comportamento foi observado nos experimentos com Vision Transformer, em que a versão simples superou as configurações que utilizaram L2, Data Augmentation e uma arquitetura mais complexa.

Esse resultado indica que, para as condições adotadas neste trabalho, as técnicas de regularização não proporcionaram ganhos de desempenho. Uma possível explicação é que essas técnicas tornam o treinamento mais conservador, exigindo um maior número de épocas para que o modelo alcance seu melhor desempenho. Como todos os experimentos foram treinados por apenas **20 épocas**, é possível que os modelos mais complexos não tenham convergido completamente.

Além disso, as matrizes de confusão mostraram um comportamento semelhante em todos os experimentos. A classe **No Tumor** foi a mais facilmente identificada pelos modelos, enquanto **Glioma** e **Meningioma** concentraram a maior parte dos erros de classificação. Esse resultado sugere que essas duas categorias apresentam características visuais semelhantes, tornando sua diferenciação mais difícil para os modelos avaliados.

6. Qual foi a melhor abordagem?

A melhor abordagem foi a MLP simples (Experimento 1), que apresentou acurácia de 81,69% e F1-Score de 0,8127. Entre os modelos baseados em Vision Transformer, o melhor desempenho foi obtido pelo Experimento 5 (Transformer simples), com F1-Score de 0,7843.

6.1 Por que as versões simples venceram?

Os resultados mostraram que as versões mais simples dos modelos obtiveram melhor desempenho em comparação com as versões que utilizaram técnicas adicionais de regularização. Tanto na MLP quanto no Vision Transformer, a inclusão de **Dropout**, **regularização L2**, **Batch Normalization** e **Data Augmentation** não resultou em melhorias durante o treinamento realizado.

Uma possível explicação é que essas técnicas tornam o processo de aprendizado mais gradual, exigindo um maior número de épocas para que seus benefícios sejam percebidos. Como todos os experimentos foram treinados por apenas **20 épocas**, é possível que os modelos mais complexos não tenham alcançado seu melhor desempenho.

Além disso, o conjunto de dados utilizado possui uma quantidade equilibrada de imagens, o que pode ter favorecido arquiteturas mais simples, capazes de aprender os principais padrões sem a necessidade de uma regularização mais intensa.

7. Conclusão

Os resultados mostraram que a MLP simples apresentou o melhor desempenho, alcançando 81,69% de acurácia e F1-Score de 0,8127. Entre os modelos Vision Transformer, o melhor resultado foi obtido pela versão simples, com F1-Score de 0,7843. De forma geral, as versões mais simples superaram as configurações com técnicas adicionais de regularização durante as 20 épocas de treinamento utilizadas neste trabalho. Além disso, foi possível observar que as classes Glioma e Meningioma apresentaram maior dificuldade de classificação, enquanto a classe No Tumor obteve os melhores resultados em praticamente todos os experimentos.

Por fim, todos os objetivos propostos foram alcançados, permitindo comparar diferentes arquiteturas e analisar o impacto das variações de parâmetros no desempenho dos modelos.

7.1 Vídeo explicativo

https://drive.google.com/file/d/1u0E88RqxQs5i_huQghyYioX60LC52kF/view?usp=sharing

7.2 Referências

- NICKPARVAR, M. Brain Tumor MRI Dataset. Kaggle, 2021.
- DOSOVITSKIY, A. et al. An Image is Worth 16x16 Words. arXiv:2010.11929, 2020.
- Documentação TensorFlow/Keras: tensorflow.org
- Documentação Scikit-Learn: scikit-learn.org